

Practical: 1

29/07/2026

Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Aim: To Implement and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

Algorithm:

Algorithms :-

i) Bubble sort :-

Step 1: for $i=0$ in condition $i < n$ then do

Step 2: for $j=0$ in condition of $j < n-i-1$ then do

Step 3: if $arr[i] > arr[j+1]$ then (swap)

swapping

$temp = arr[i]$

$arr[i] = arr[j+1]$

$arr[j+1] = temp$

Step 4: increment the inner loop of $j++$ iteration

Step 5: increment the outer loop of $i++$ iteration

Step 6: Stop

ii) Selection

Step 1: for $i=0$ in the condition $i < n$ then do

Step 2: $min = i$ and inner for loop initialize $j = i+1$

Step 3: in the condition $j < n$ then do

if $arr[j] < arr[min]$ then

$min = j$

Step 4: increment the inner loop $j++$

Step 5: if $min \neq i$ then

Swap ($arr[i], arr[min]$)

Step 6: increment the outer loop $i++$

Step 7: Stop

iii) Insertion sort :-

Step 1: for $i=1$ in the condition $i < n$ then do

Step 2: for $temp = arr[i]$ and $j = i-1$

Step 3: while $j >= 0$ and $arr[j] > temp$ then do

$arr[j+1] = arr[j]$

$j = j-1$

Step 4: initialize $arr[j+1] = temp$

Step 5: increment the outer loop $i++$

Step 6: Stop

iv) Merge Sort

Step 1: check if array has more than one element

Step 2: Divide the array into two equal halves

Step 3: Recursively sort the left half using merge sort

Step 4: Same for right half using merge sort

Step 5: merge the two sorted halves into a single sorted array

Step 6: Stop

v) Quick sort :-

Step 1: Select any element as a pivot

Step 2: partition the array by placing

Step 3: if $arr[j] \leq pivot$

$i++$

Step 4: place pivot in its correct sorted position

Step 5: Recursively apply Quick Sort

Step 6: left sub-array and right sub-array

Step 7: Stop

Program:**1. Bubble sort, selection sort, Insertion sort**

```
#include<iostream>
using namespace std;

class classname{
public:
    void Func(){
        int array1[100];
        int input,i;
        cout<<"Enter the size of array: ";
        cin>>input;
        array1[input];
        cout<<"Enter the Elements in array:\n";
        for(int i=0; i< input; i++){
            cin>>array1[i];
        }
        int n = sizeof(array1)/sizeof(array1[0]);
        display(array1,input);
        cout<<"\n\n";
        Bubble(array1,input);
        cout<<"\n\n";
        selection(array1,input);
        cout<<"\n\n";
        insertion(array1,input);
        cout<<"\n\n";
        // quickSort(array1, 0, n-1);
    }
    void display(int array1[],int n){
        cout<<"Your Array is:\n";
        for(int i = 0; i<n; i++){
            cout<<array1[i]<<" ";
        }

    }
    void Bubble(int array1[],int n){
        cout<<"Bubble sorted array:\n";
        for(int i=0;i<n;i++){
            for(int j=0; j<n-i-1; j++){
                if(array1[j] > array1[j+1]){
                    int temp = array1[j];
                    array1[j] = array1[j+1];
                    array1[j+1] = temp;
                }
            }
        }
    }
```

```
        }
        display(array1,n);
    }
    void selection(int array1[],int n){
        cout<<"selection sort Array:\n";
        for(int i = 0; i<n; i++){
            int min = i;
            for(int j=i+1; j<n; j++){
                if(array1[j] < array1[min]){
                    min = j;
                }
            }
            if(min != 0){
                int temp = array1[i];
                array1[i] = array1[min];
                array1[min] = temp;
            }
        }
    }
    display(array1,n);
}

void insertion(int array1[],int n){
    cout<<"insertion sort Array:\n";
    for(int i=1;i<n;i++){
        int temp = array1[i];
        int j = i-1;
        while(j>=0 && array1[j]>temp){
            array1[j+1] = array1[j];
            j=j-1;
        }
        array1[j+1] = temp;
    }
    display(array1,n);
}

int main(){
    classname obj;
    obj.Func();
    return 0;
}
```

2. Quick sort and merge sort:

```
#include<iostream>
using namespace std;
```

```
//----- Swap Function -----
```

```
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

//----- Quick Sort -----
int partition(int arr[], int start, int end) {
    int pivot = arr[end];
    int i = start - 1;

    for (int j = start; j < end; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[end]);
    return i + 1;
}

void quickSort(int arr[], int start, int end) {
    if (start < end) {
        int pi = partition(arr, start, end);

        quickSort(arr, start, pi - 1);
        quickSort(arr, pi + 1, end);
    }
}

//----- Merge Sort -----
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    // Copy data to temporary arrays
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
```

```
// Merge the temporary arrays
while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
    }
    else {
        arr[k] = R[j];
        j++;
    }
    k++;
}

// Copy remaining elements
while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}

}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

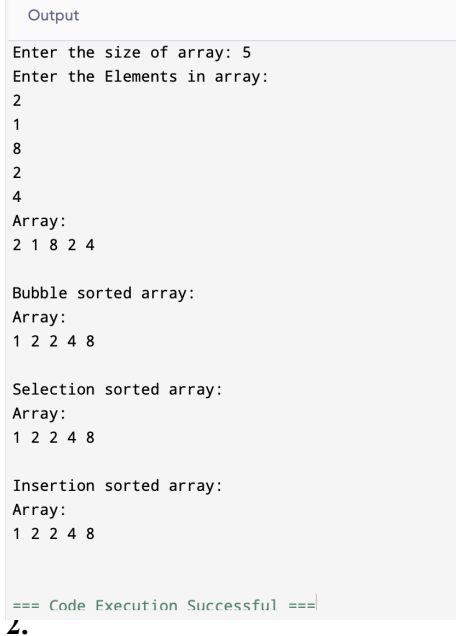
void display(int array1[], int n) {
    for (int i = 0; i < n; i++) {
        cout << array1[i] << " ";
    }
    cout << endl;
}

int main() {
    int arr[] = {9, 9, 7, 78, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
```

```
cout << "Original Array:\n";
display(arr, n);
//Quick Sort
quickSort(arr, 0, n - 1);
    cout<<endl;
cout<<"Quicksort sorted array: ";
display(arr, n);
// Merge Sort
mergeSort(arr, 0, n - 1);
cout<<endl;
cout<<"mergesort sorted array: ";
display(arr, n);
return 0;
}
```

Output:

1



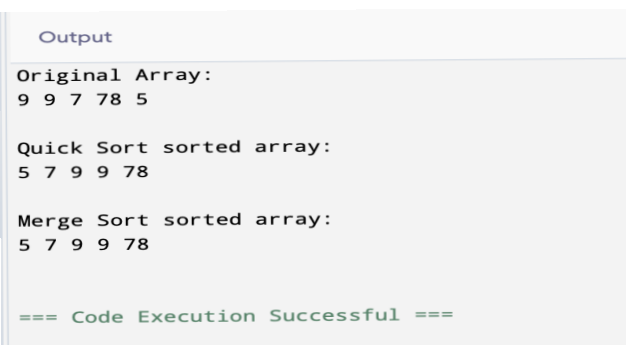
```
Output
Enter the size of array: 5
Enter the Elements in array:
2
1
8
2
4
Array:
2 1 8 2 4

Bubble sorted array:
Array:
1 2 2 4 8

Selection sorted array:
Array:
1 2 2 4 8

Insertion sorted array:
Array:
1 2 2 4 8

=== Code Execution Successful ===
```



```
Output
Original Array:
9 9 7 78 5

Quick Sort sorted array:
5 7 9 9 78

Merge Sort sorted array:
5 7 9 9 78

=== Code Execution Successful ===
```

Time Complexity:

Time Complexity:

1) **Bubble sort:**

```

for(i=0; i<n-1; i++) {
    for(j=0; j<n-i-1; j++) {
        if(arr[j] > arr[j+1]) {
            t = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = t;
        }
    }
}

```

Total Tcm = $(n-1) + (n-2) + \dots + 1$
 $= \frac{n(n-1)}{2} + n - 1$
 $= \frac{n^2 - n}{2} + n - 1$
 $= \frac{n^2 - n + 2n - 2}{2}$
 $= \frac{n^2 + n - 2}{2}$
 $\approx \frac{n^2}{2}$
Time Complexity = $O(n^2)$

2) **Selection sort:**

```

for(i=0; i<n-1; i++) {
    min = i;
    for(j=i+1; j<n; j++) {
        if(arr[j] < arr[min]) {
            min = j;
        }
    }
    if(min != i) {
        t = arr[i];
        arr[i] = arr[min];
        arr[min] = t;
    }
}

```

$\approx 2n + 2 + n + \frac{3}{2}(n(n-1))$
 $\approx 3n^2 + 3n + 2 \Rightarrow O(n^2)$

3) **Insertion sort:**

```

for(i=1; i<n; i++) {
    t = arr[i];
    j = i-1;
    while(j > 0 & arr[j] > t) {
        arr[j+1] = arr[j];
        j = j-1;
    }
    arr[j+1] = t;
}

```

$\approx 2n + 2(n-1) + \frac{n(n-1)}{2} + (n-1)$
 $\approx 3n - 2 + 2n + \frac{n^2 - n}{2} + n - 1$
 $\approx 3n - 2 + 2n + \frac{n^2 - n}{2} - 1$
 $\approx 5n - 3 + \frac{n^2 - n}{2}$
 $\approx \frac{n^2}{2} + 4.5n - 3$
Time Complexity = $O(n^2)$

4) **Merge sort:**

```

mergeSort(arr, left, right) {
    if (left < right) {
        m = (left + right) / 2;
        mergeSort(arr, left, m);
        mergeSort(arr, m+1, right);
        merge(arr, left, m, right);
    }
}

```

$T(n) = O(n) \times O(\log_2 n)$
 $= O(n \log n)$

Time Complexity:

1) **Bubble sort:**
 Best case: $O(n)$
 Average case: $O(n^2)$
 Worst case: $O(n^2)$

2) **Selection sort:**
 Best case: $O(n^2)$
 Average case: $O(n^2)$
 Worst case: $O(n^2)$

3) **Insertion sort:**
 Best case: $O(n)$
 Worst case: $O(n^2)$
 Average case: $O(n^2)$

4) **Merge sort:**
 Best case: $O(n \log n)$
 Worst case: $O(n \log n)$
 Average case: $O(n \log n)$

5) **Quick sort:**
 Best case: $O(n \log n)$
 Average case: $O(n \log n)$
 Worst case: $O(n^2)$

Quick sort:

```

quickSort(arr, s, e) {
    if (s < e) {
        p = partition(arr, s, e);
        quickSort(arr, s, p-1);
        quickSort(arr, p+1, e);
    }
}

```

$T(n) = O(n) \times O(\log_2 n)$
 $T(n) = O(n \log n)$

$T(n) = T(n-1) + O(n)$
 $T(n-1) = T(n-2) + O(n-1)$
 $T(n) = T(n) + 2 + 3 + 4 + \dots + n$
 $T(n) = O(n^2)$

Conclusion:

Hence, By Implementing and analysing of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

The time complexity plays an important role in the logical part of the program. And the Programs are successfully executed.